

VectorLLM: Language-Controllable Autonomous Driving via Vectorized Scene Grounding

Author One
Affiliation
City, Country
author1@example.com

Abstract: Natural language provides a flexible interface for humans to interact with autonomous vehicles. However, the effectiveness of this interaction relies on a driving model’s language *grounding* and *controllability*. Recent works have leveraged vision-language models (VLMs) as backbones but are still limited in grounding driving scenes with language. We present VectorLLM, a vectorized paradigm that grounds vectorized driving scenes with pretrained large language models (LLMs). The vectorized agent motions and map elements are represented as discrete tokens and learned to align with language in LLM’s latent space. This approach directly incorporates driving information into LLMs to utilize the pre-trained LLM capabilities, e.g., chain-of-thought reasoning and language generation, to reason about and plan in a vectorized driving scene. We conduct extensive experiments with Waymo Open Motion dataset and nuScenes dataset across multiple tasks, including trajectory prediction, question answering, and controllability under counterfactual scenes and instructions. We show that VectorLLM can leverage low-level driving information to generate trajectories and outperform existing LLMs/VLMs in WOMD-Reasoning and nuScenes-QA, demonstrating improved grounding. More importantly, VectorLLM demonstrates superior controllability compared to alternative models when responding to the changes in scenes and counterfactual language instructions, offering an effective language interface for controllable driving interactions.

Keywords: LLM-based Driving, Vectorized Scene Representation, Language Grounding, Language Control

1 Introduction

Natural language offers a flexible interface for human-vehicle interaction, allowing a vehicle to explain its behavior, answer questions about driving scenes, and adapt its maneuvers in response to human instructions. For such interaction to be effective, a driving model cannot just generate fluent sentences or predict a road agent’s trajectory; it must accurately *ground* language in the scene and ensure its maneuvers are reliably *controllable* [1] by language. Recent driving LLM and VLM systems have explored this direction by representing driving scenes through 2D images/videos [2, 3, 4, 5], object-level embeddings [6], or coarse textual descriptions [7, 8, 9, 10]. While these modalities provide useful semantic context, prior studies have shown that these models are limited in precise scene grounding (e.g., spatial reasoning [11, 12], vehicle dynamics [13], and agent interactions [14]) and language controllability [15, 16].

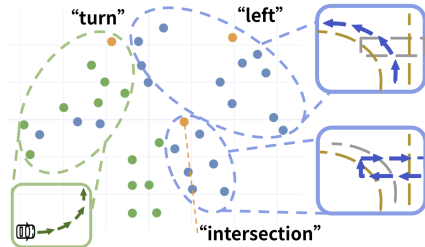


Figure 1: **Alignment between language and vector tokens in latent space.** t-SNE plot of token embeddings from a language-conditioned trajectory generation task “turn left at the intersection”. Language (orange) tokens are close to its corresponding map (blue) and trajectory (green) vector tokens, circled in dashed lines.

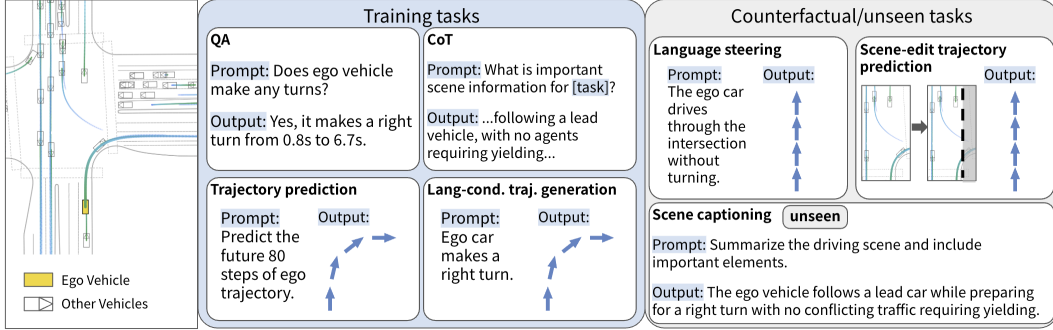


Figure 2: **Overview of VectorLLM’s task capabilities.** Given a driving scene, VectorLLM not only performs the tasks it was trained on, but also handles counterfactual scenarios and an unseen scene captioning task. The example prompts and language/vector outputs illustrate its unified reasoning, prediction, and generalization abilities.

To address these limitations, we introduce **VectorLLM**, a vectorized paradigm for integrating driving scenes with pretrained LLMs in a unified token sequence. VectorLLM discretizes vectorized scene elements into *vector tokens* and serializes them with language. This tokenized interface maps structured map and motion information into the language-model token space while preserving precise geometric cues, including road geometry, lane boundaries, and traffic flow, which are essential for effective end-to-end planning [17, 18, 19]. Through fine-tuning, VectorLLM aligns vector tokens with language in the LLM’s latent space (Fig. 1), allowing a single model to reason about traffic context, answer questions, and generate trajectories directly from low-level driving scenes.

The vector-token representation also supports generalization to new driving-language tasks. We observe that the model can caption vectorized scenes (Fig. 2) even though the model was not trained on captioning tasks, demonstrating that pretrained knowledge can successfully ground itself in vector tokens. Furthermore, this representation enables *controllable trajectory generation*, allowing VectorLLM to condition generated trajectories on modified scene elements or counterfactual language instructions (Fig. 2).

We train and evaluate VectorLLM extensively on the Waymo Open Motion Dataset (WOMD) [20] and nuScenes [21] across four tasks: trajectory prediction, question answering (QA), scene-edit controllability, and instruction controllability. This work makes these contributions:

- We introduce VectorLLM, a paradigm that represents vectorized driving scene elements and language prompts as a unified token sequence, enabling pretrained LLMs to natively reason over structured driving scenes.
- We demonstrate that aligning vector and language tokens yields improved language grounding, outperforming existing LLM/VLM baselines on question answering tasks in WOMD-Reasoning [22] and nuScenes-QA [11].
- We show that VectorLLM achieves highly controllable behavior that adapts to changes in scenes and follows counterfactual language instructions while generating trajectories comparable to specialized models on WOMD and nuScenes.

2 Related Work

LLM/VLM-based Driving Recent works leverage LLMs and VLMs as planning or reasoning modules for autonomous driving. A common design encodes multi-view images into visual tokens [23, 4, 24, 3, 5] and uses structured reasoning to convert visual scene understanding into meta-actions, explanations, or trajectories. However, precise driving-scene grounding remains challenging for visual-token interfaces, especially for 3D structure and agent interactions, motivating additional mechanisms for improving geometric reasoning [5, 12]. Another line of work converts structured scene information into textual prompts [10, 8, 9]. These methods provide LLMs with simplified abstractions of map elements, agent states, or traffic rules, but the textual interface compresses the scene and makes performance depend on whether the prompt includes task-relevant details. In con-

trast, VectorLLM represents map geometry and agent motion as discrete tokens, enabling the LLM to reason jointly over geometric structure and language.

Structured Driving Scene Representations Explicitly modeling driving scenes with vectorized [17, 18] and interaction- or graph-based [25, 26, 27] representations has been effective for trajectory forecasting and planning. Recent works further show that driving scenes can be tokenized and modeled as discrete sequences for trajectory prediction [28, 19]. VectorLLM builds on this line of structured scene modeling, but differs in its goal and interface: rather than training a specialized forecasting model over scene tokens alone, we integrate vectorized map and motion tokens into a pretrained LLM vocabulary and use the resulting unified token sequence for language-conditioned reasoning, question answering, and trajectory generation.

Language Grounding and Control in Driving Natural language integration in autonomous driving requires grounding instructions in relevant scene elements and generating behavior that follows them reliably. Recent driving QA benchmarks [11, 22, 14, 24] highlight challenges in spatial, interaction, temporal, and counterfactual reasoning, while prior control-oriented works use diffusion-guided constraints [29], text-based scenario editing [30], or instruction-paired trajectories [31, 32, 33]. Among them, iMotion-LLM [33] is closest to ours, as it also uses vectorized driving-scene information with an LLM for instruction-conditioned trajectory generation. Unlike iMotion-LLM, which uses a trajectory-prediction encoder to project continuous scene features into the LLM input space, VectorLLM represents map and motion displacements as discrete tokens in the LLM vocabulary. By serializing these tokens directly with language prompts, VectorLLM removes the need for separate scene encoders or projection modules while keeping scene elements explicitly token-addressable for grounded reasoning and controllable generation.

3 Problem Statement

We consider multi-task learning for autonomous driving within a language modeling framework. Our goal is to learn an LLM $\mathcal{M}(O|L_{\text{in}}, S)$ that maps the language prompt L_{in} , consisting of a task specification, and an input driving scene S , to generate task-specific output O . For tasks like prediction and planning, the output is trajectories O_{traj} that the vehicle can follow. For tasks like QA and captioning, the output is reasoning trace and language responses O_{lang} . The information the LLM can access is constrained by the types of tokens and the size of its input context window.

Prior driving vision-language-action models (VLAs) often rely on separate encoders or adapters to convert maps, agent motions, image features, or task-specific inputs into continuous embeddings for the language model. Such models learn over multimodal prompts, but the grounding between language and driving scene is mediated through these learned feature embeddings. However, in this setting, scene elements (e.g., map and agent motions) are not necessarily represented in the learned features for the LLM to access which makes it fundamentally harder to analyze what language grounds. To address this issue, we explore an alternative approach to represent driving scenes as discrete tokens that can be directly integrated with language tokens. Let $\mathcal{V}_s$ be the vocabulary used to describe the driving scene elements. We expand the LLM’s original language vocabulary $\mathcal{V}_l$ with the scene vocabulary $\mathcal{V} = \mathcal{V}_l \cup \mathcal{V}_s$. Thus, the input scene S and output trajectories O_{traj} are tokenized using $\mathcal{V}_s$, allowing the expanded multimodal sequence to be processed by a single LLM backbone.

4 Method

Fig. 3 shows the overall architecture of the VectorLLM. A driving scene is first vectorized and then discretized into a series of tokens. These vector tokens are concatenated with language tokens to construct the LLM’s input context. The VectorLLM employs a pretrained decoder-only LLM as the backbone. We use a bidirectional attention mask for trajectory output to account for the temporal structure of trajectory reasoning while maintaining standard causal attention elsewhere. We train the model with chain-of-thought reasoning. Notably, the reasoning trace explicitly selects and references relevant scene elements using vector tokens, ensuring the language generation is grounded in the driving scene. The output tokens are decoded into language answers, intermediate goals, or trajectory vectors depending on the task.

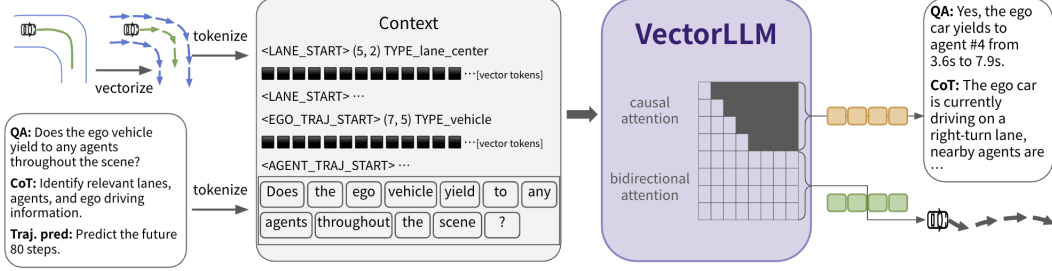


Figure 3: **Overview of VectorLLM.** (1) VectorLLM tokenizes vectorized map geometry, ego- and ado-agent motions, and language prompts into the LLM’s input context. (2) A decoder-only LLM backbone processes this expanded token sequence to support multiple driving tasks. Chain-of-thought supervision is employed to enhance performance and explainability. (3) In addition to language QA, we highlight the capabilities in trajectory generation.

4.1 Vectorization and Tokenization of Driving Scenes

Driving Scene Vectorization VectorLLM takes vectorized driving-scene elements as input. For static map context, these elements come from either HD maps or MapTR-predicted vectorized map elements from multi-view images [34].

This input scene comprises the coordinates for the static map elements (e.g., lanes, road edges, cross-walk, and stop signs) and the dynamic agent motions (including the trajectory history of the ego and ado agents). To ensure alignment across scenes, all coordinates are transformed into the normalized frame of the ego agent. The key step is to express both map elements and agent motions within the same vectorization scheme: an ordered sequence of two-dimensional displacement vectors. For an agent trajectory with positions $\{p_t\}_{t=1}^{t=T}$, each raw motion vector is $v_t = p_t - p_{t-1} = (\Delta x_t, \Delta y_t)$. For static map elements, the same operation is applied spatially along the road graph structure: map polylines are re-sampled into ordered points using curvature-adaptive step sizes: shorter vectors are used in high-curvature regions and longer vectors in near-linear regions, with resolutions of 0.5m, 1.0m, 1.5m, and 2.0m. Each raw road vector then represents the spatial displacement $(\Delta x, \Delta y)$ between adjacent sampled points.

Tokenization These continuous vectors are discretized into tokens. To build this vocabulary, we collect per-step agent displacement vectors from the WOMD training split and apply k-disks clustering [28] to extract representative centroids. Each centroid is assigned one special vector token, denoted VEC_i . This creates a scene vocabulary $\mathcal{V}_s$ that we use to represent both temporal agent motions and static map structures. Since a vector token only encodes local displacement, it lacks driving semantics such as whether a token sequence belongs to the ego vehicle or a lane. To provide this context, we introduce high-level semantic marker tokens and delimiter tokens to indicate the beginning and end of each type of token sequence (see Appendix A for the complete list of special tokens). During tokenization, since naive nearest-centroid assignment can accumulate reconstruction drift over long road polylines, we employ a beam-search assignment procedure that selects a sequence of vector tokens to minimize the cumulative displacement error of the reconstructed polyline. We empirically validate that this procedure mitigates cumulative drift, reconstructing road polylines with an error below 0.03m in WOMD. See Appendix A for more details.

LLM Input Context After tokenization, the driving scene is serialized as a single token sequence to combine with language prompts. A representative context structure includes these blocks:

$$[\text{MAP}, \text{ROAD}_0, \text{ROAD}_1, \dots, \text{EGO_TRAJ}, \text{AGENT_TRAJ}_1, \text{AGENT_TRAJ}_2, \dots, q, \text{TRJ_MARKER}]$$

where q is an optional language question, instruction, or auxiliary prompt. The actual map and agent blocks contain their semantic marker tokens and delimiter tokens when available, and the corresponding VEC sequences. The map blocks typically occupy the largest fraction of the context because road geometry contains many spatial vectors. During training, scenes longer than the N_c context window size are dropped. At inference time, we do not use this same hard filter, since the practical constraint is GPU memory rather than the nominal context length.

4.2 Model Architecture

LLM Backbone VectorLLM builds upon a pretrained decoder-only LLM by expanding its vocabulary to incorporate vector tokens. Rather than repurposing existing tokens, we preserve the original language vocabulary and expand both the input embedding matrix and the output language-modeling head to include the new vector and special tokens. These newly added embeddings are initialized using the mean and covariance of the pretrained embedding distribution, while existing language-token embeddings are not overwritten. Our model architecture relies purely on the LLM backbone. There is no need for a separate map encoder, motion encoder, or intermediate projection layer. VectorLLM is trained by full fine-tuning: the pretrained transformer parameters, original language token embeddings, newly added token embeddings, and expanded output head are all trainable.

Chain-of-thought (CoT) Reasoning VectorLLM employs CoT reasoning to explicitly align language with vector tokens before committing to a task output. The CoT traces follow a driving-oriented reasoning flow: “identify the ego motion state → select relevant map context and nearby agents → reason about interactions or feasible maneuvers → support the final answer or trajectory generation.” For QA, we construct the trace to focus only on evidence relevant to the question. For trajectory generation, we construct the trace using a broader scope of future driving options, e.g., turning or yielding, to explain the intended maneuver. To improve reliability for long-horizon planning, all trajectory generation tasks in VectorLLM utilize goal-conditioned generation similar to [35, 36]. Following the CoT rationale, the model first predicts three sparse future waypoints to represent ego’s goal. This goal is not decoded as a discrete token sequence; instead, a lightweight MLP head reads the LLM hidden states at designated goal positions to regress continuous metric coordinates. The vector token trajectory is then generated conditioned on this intermediate goal.

Decoding Strategies VectorLLM uses different decoding strategies for language and trajectory generation. For language outputs, we preserve the standard autoregressive decoding with causal attention. Trajectory outputs are generated using parallel decoding similar to OpenVLA-OFT [37]. The designated goal and trajectory output positions are initialized with zero embeddings in the LLM input context. Bidirectional attention is applied only to these target positions. Under this masking scheme, target positions can attend to all scene and prompt tokens in the context, and they can also attend to every other target position. Context tokens use causal attention and are prevented from attending to target positions. This design gives one-way information flow from the serialized scene context to the target positions while allowing all future motion positions to be decoded jointly in a single pass. Finally, the expanded language-modeling head produces a distribution over the vector tokens for each target position. The selected tokens are mapped back to their displacement centroids and reconstructed into metric coordinates by cumulative summation.

4.3 Training Pipeline

VectorLLM is trained using multiple stages, with all model parameters remaining trainable in every stage. In Stage I, we perform vector token pretraining. In this stage, the model performs standard next-token prediction over the serialized driving scenes. This stage is used only to expose the pretrained LLM to learn map and motion structures as ordinary token sequences. In Stage II, we perform multi-task fine-tuning on a mixture of downstream tasks: driving question answering, goal prediction, and trajectory generation. The training objective combines the losses active for each task. Both vector token pretraining and the language outputs are supervised with standard autoregressive cross-entropy over the expanded vocabulary. Trajectory generation employs cross-entropy over the vector tokens at each trajectory target position. The intermediate goal positions are supervised using an ℓ_2 loss applied to the MLP goal head.

5 Experiments

We conduct extensive experiments on WOMB and nuScenes to address these research questions.

- RQ1** How effectively does the shared token space ground language in driving scenes?
- RQ2** Does token alignment translate to controllable driving behavior?
- RQ3** Can serialized vectorized scenes serve as an effective token interface for pretrained LLMs to generate and reason about driving behavior?

5.1 Experimental Setup

Implementation We use Qwen3-0.6B [38] as the LLM backbone, though the architecture is compatible with any pretrained decoder-only LLM. While the standard VectorLLM setting uses HD-map inputs, we also evaluate a perception-based variant, VectorLLM+MapTR, in which MapTR [34] converts multiview images into vectorized map elements that are then tokenized and provided to VectorLLM. This perception-based variant probes VectorLLM under noisy scene elements from a realistic perception front end, while also enabling comparison with perception-input VLM/VLA systems. Our scene vocabulary is set to $|\mathcal{V}_s| = 1024$ vector tokens. During training, we limit the input context to $N_c = 12288$ tokens; dropping scenes that exceed this length removes just 0.6% of the training scenes from WOMD.

Training Data We use WOMD training split as our primary data source for VectorLLM pretraining and multi-task fine-tuning. WOMD provides map elements, agent histories, and future trajectories in a format that can be vectorized and converted to VectorLLM’s inputs. For multi-task fine-tuning, to construct task-specific language data, e.g., QA, CoT, and language instructions, we first apply rule-based filters to extract information from the driving scenes. We then prompt Qwen3-4B-Instruct-2507 to synthesize natural language sentences based on this extracted data. Details regarding prompts, templates, and task formatting are provided in the Appendix B.

Evaluation Benchmarks We evaluate VectorLLM on benchmarks covering trajectory generation, language grounding, and controllability. For trajectory generation, we test marginal trajectory prediction on the WOMD validation split [20] and nuScenes [21]. For language grounding, we evaluate driving question answering on WOMD-Reasoning [22] and nuScenes-QA [11]. We further introduce two custom counterfactual test sets to measure controllability under modified scenes and counterfactual language instructions. Including nuScenes evaluates transfer beyond WOMD, while VectorLLM+MapTR allows us to assess performance under noisy map inputs and directly compare with perception-based input representations. Before evaluation, VectorLLM is fine-tuned on the designated training set for each benchmark. See Appendix C for metrics definitions.

5.2 Language Grounding (RQ1)

We evaluate VectorLLM’s language grounding capability through QA. If the model correctly relates language tokens to the corresponding scene elements represented by vector tokens, it can retrieve the relevant information to answer questions.

WOMD-Reasoning Table 1 compares VectorLLM against Motion-LLaVA, a motion-language baseline from [22]. VectorLLM improves over Motion-LLaVA on both accuracy within a 1.0-meter tolerance ($\text{Acc}_{1.0}$) and language metrics (ROUGE).

This suggests that explicitly representing scenes as vector token sequences allows LLMs not only to identify relevant information but also to generate high-quality textual responses.

nuScenes-QA nuScenes-QA tests language grounding using questions built from the nuScenes scene graph. Table 2 shows that VectorLLM improves most on questions that require no reasoning between scene elements, demonstrating the direct effect of improved grounding. See Appendix C for the complete question-type breakdown. The VectorLLM+MapTR variant performs closely to the perception-based baseline, OpenDriveVLA [12], showing the feasibility of using vectorized maps produced by a perception front-end, even though the perception stage introduces additional noise.

Table 1: QA performance on WOMD-Reasoning.

Method	$\text{Acc}_{1.0}\uparrow$	ROUGE $\uparrow$
Motion-LLaVA [22]	56.2%	0.792
VectorLLM	61.0%	0.804

Table 2: Performance on nuScenes-QA, split by zero-hop (H0) and one-hop (H1) questions.

Method	Modality	H0 $\uparrow$	H1 $\uparrow$	Acc. $\uparrow$
LLaMA-AdapV2 [39]	Perception	15.1	4.8	9.6
LLaVA1.5 [40]	Perception	25.7	41.5	26.2
OpenDriveVLA-0.5B [12]	Perception	62.3	56.5	58.4
VectorLLM	HD map	69.9	60.4	63.5
VectorLLM+MapTR	Perception	61.7	55.9	57.8

5.3 Controllability of Driving Behavior (RQ2)

We hypothesize that improved grounding enables controllable trajectory generation under changes in either scene or language. Since existing datasets pair static scenes with single fixed futures, they cannot rigorously test counterfactual responsiveness. We therefore create two custom benchmarks

from WOMB and nuScenes; see Appendix C for details. By editing scene elements, such as lane geometry, add agents, or ego history, or by providing alternative language commands, such as turning instead of going straight, we test whether the model steers toward the intended counterfactual behavior. Steering success is measured by goal hit rate with a 1.0m tolerance over six sampled futures at manually labeled target horizons: 3s, 5s, and 8s for WOMB, and 1s, 2s, and 3s for nuScenes.

Table 3: Trajectory generation goal hit rate in counterfactual edited scenes, split by edit types on WOMB.

Method	Lane ↑	Agent↑	Ego History↑	All↑
MTR [36]	52.0%	53.0%	62.0%	55.0%
M2I [27]	36.0%	37.0%	20.0%	32.5%
VectorLLM	68.0%	58.0%	66.0%	62.5%

Table 4: Trajectory generation goal hit rate with counterfactual language instructions at different time horizons on nuScenes.

Method	w/ Lang.	Avg.↑	@1s↑	@2s↑	@3s↑
VectorLLM	✗	31.8%	53.0%	33.5%	9.0%
CTG++ [29]	✓	14.0%	30.5%	10.5%	1.0%
iMotion-LLM [33]	✓	67.5%	83.5%	68.0%	51.0%
VectorLLM	✓	75.4%	91.0%	73.5%	62.0%

Counterfactual Scene Edits We created counterfactual scene edits from the WOMB to evaluate VectorLLM against strong pretrained trajectory prediction models, MTR [36] and M2I [27]. Table 3 shows that VectorLLM outperforms these baselines across all scene edit types. The performance gains are most prominent in lane edits or when adding agents, where the intervention directly changes local map affordances or interaction constraints.

Counterfactual Language Instructions We created counterfactual language instructions from nuScenes to compare VectorLLM with CTG++ [29], a pretrained diffusion-based generation model with language guidance, and iMotion-LLM [33], an LLM baseline using vectorized scene inputs that we train for this setting. Table 4 reports the results, including unconditional VectorLLM as a reference. VectorLLM significantly outperforms the baselines across all horizons and degrades less at longer horizons, demonstrating stronger controllability over extended sequences.

5.4 Serialized Vectorized Scenes as an LLM Token Interface (RQ3)

We analyze whether the serialized vector tokens provide an effective interface between driving scenes and pretrained LLMs. Such an interface should 1) preserve sufficient low-level information for trajectory generation while 2) allowing the LLM to reason over vector token sequence. We evaluate both properties by comparing VectorLLM’s trajectory generation against specialized models and ablating CoT.

WOMB Trajectory Forecasting Table 5 summarizes the performance compared to existing specialized forecasting models. While VectorLLM’s results trail the strongest specialized forecasting model, MTR [36], its performance is comparable or even outperforms established baselines such as MultiPath [41] and DenseTNT [35]. This demonstrates that the vector token representation retains low-level information to support trajectory prediction, rather than only producing language outputs.

Table 5: WOMB trajectory prediction results on the standard validation set with vehicle only.

Method	minADE↓	MR↓	mAP↑
LSTM [20]	1.34	0.25	0.23
MultiPath [41]	1.00	0.30	0.26
DenseTNT [35]	1.35	0.15	0.37
Scene Transformer [42]	1.17	0.20	0.26
MTR [36]	0.76	0.15	0.45
VectorLLM	0.92	0.25	0.34

nuScenes Motion Forecasting We assess cross-dataset applicability on nuScenes using the ST-P3 motion forecasting protocol [44], which evaluates marginal prediction of the future ego trajectory. Although our scene serialization is derived from Waymo-style vectors and tokens, we show that the same representation can be adapted to another dataset by fine-tuning on nuScenes. Table 6 shows that VectorLLM performs comparably to VAD [18]. While OpenDriveVLA [12] reports better performance, it uses additional ego-state cues such as speed and steering. VectorLLM+MapTR shows an expected drop from using noisy perception-predicted map elements, but still remains competitive with several perception-input baselines. This suggests that the vector-token interface can tolerate imperfect scene maps to some extent.

Table 6: nuScenes motion forecasting performance comparison.

Method	L2↓	Col.↓
VAD [18]	0.37	0.14
UniAD [43]	0.69	0.12
OmniDrive [24]	0.33	0.30
OpenDriveVLA-0.5B [12]	0.35	0.09
VectorLLM	0.39	0.17
VectorLLM+MapTR	0.45	0.20

CoT Ablation An effective token interface should enable the LLM to reason over the serialized scenes. To verify this, we compare VectorLLM against two ablations: 1) removing the CoT to directly output the answer or trajectory and 2) corrupting the reasoning traces with inconsistent or irrelevant information. This setting tests whether the performance gains come from intermediate reasoning rather than simply generating more tokens. As shown in Table 7, accurate CoT improves both trajectory prediction and QA, while corrupted or no CoT significantly underperform the full model. We include qualitative examples of reasoning chains in Appendix C.

5.5 Qualitative Analysis

To understand how VectorLLM grounds language and vector tokens in latent space, we visualize token attention (Fig. 4) and the t-SNE of token embeddings (Fig. 1). Fig. 4 highlights the scene elements attended by the specific language tokens (underlined) during QA and language-conditioned trajectory generation. The attention pattern reflects token semantics. For example, “turn” strongly attends to right-turn lanes (Fig. 4(1)), while “yield” attends to all other agents (Fig. 4(2)). In the language-conditioned trajectory examples, the right-turn instruction directs attention primarily toward the turning lane and the road region associated with the intended maneuver (Fig. 4(4)). In contrast, the counterfactual instruction to drive through the intersection reduces attention on the turn lane and shifts focus toward traffic participants and activity near the intersection area (Fig. 4(5)). We include more visualization examples in Appendix C.

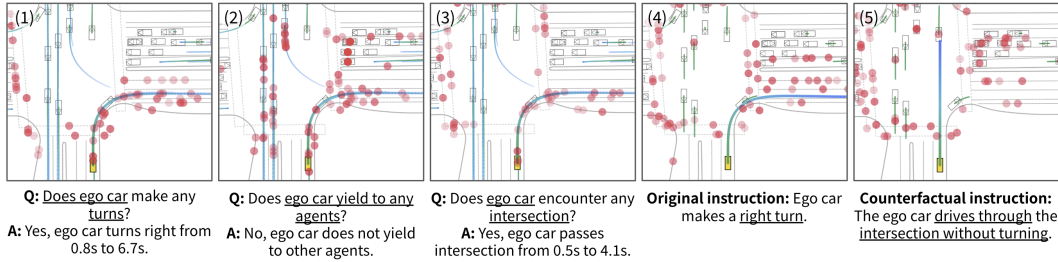


Figure 4: Attention weights are visualized by red dots, where darker dots indicate higher attention weights. The ego vehicle is shown in yellow, and surrounding agents are shown in blue.

6 Conclusion

We present VectorLLM as a unified vector-language interface for language-controllable autonomous driving. By representing map geometry and agent motion as discrete vector tokens alongside language, VectorLLM enables pretrained LLMs to ground driving-scene structure, perform reasoning, answer questions, and generate trajectories within a single framework. Experiments across WOMB and nuScenes show that this token-level alignment improves driving QA and shows superior controllability under counterfactual scene edits and language instructions. These results suggest that vectorized scene tokenization is an effective abstraction for connecting driving information with the reasoning and generation capabilities of LLMs.

6.1 Limitations

While VectorLLM shows strong grounding and controllability, several limitations remain. First, it is not co-trained with BEV map generation or raw visual perception. Although it can use maps reconstructed by a perception front-end, reconstruction noise or mismatch may affect performance. Second, QA failures suggest that the model often identifies the relevant driving behavior but struggles to localize its temporal boundaries, pointing to the need for finer-grained temporal understanding. Third, trajectory prediction errors can accumulate from early steps, especially in fast-moving scenes. Finally, counterfactual evaluation remains imperfect: when instructions require large deviations from default behavior, the model may resist steering, motivating stronger conditioning mechanisms to override behavioral priors.

Table 7: Ablation of CoT on WOMB trajectory prediction and WOMB-Reasoning QA tasks.

Setting	Traj. Pred.		QA	
	minADE↓	minFDE↓	Acc _{1.0} ↑	ROUGE↑
VectorLLM	0.92	2.06	61.0%	0.804
w/ corrupted CoT	1.19	2.54	53.7%	0.715
w/o CoT	1.31	2.75	51.2%	0.712

References

- [1] S. Gao, J. Yang, L. Chen, K. Chitta, Y. Qiu, A. Geiger, J. Zhang, and H. Li. Vista: A generalizable driving world model with high fidelity and versatile controllability. *Advances in Neural Information Processing Systems*, 37:91560–91596, 2024.
- [2] Wayve. LINGO-1: Exploring natural language for autonomous driving. <https://wayve.ai/thinking/lingo-natural-language-autonomous-driving>, September 2023. Accessed: 2026-05-26.
- [3] X. Tian, J. Gu, B. Li, Y. Liu, Y. Wang, Z. Zhao, K. Zhan, P. Jia, X. Lang, and H. Zhao. DriveVLM: The convergence of autonomous driving and large vision-language models. In P. Agrawal, O. Kroemer, and W. Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4698–4726. PMLR, 06–09 Nov 2025.
- [4] Z. Xu, Y. Zhang, E. Xie, Z. Zhao, Y. Guo, K.-Y. K. Wong, Z. Li, and H. Zhao. DriveGPT4: Interpretable end-to-end autonomous driving via large language model. *IEEE Robotics and Automation Letters*, 9(10):8186–8193, 2024.
- [5] J.-J. Hwang, R. Xu, H. Lin, W.-C. Hung, J. Ji, K. Choi, D. Huang, T. He, P. Covington, B. Sapp, Y. Zhou, J. Guo, D. Anguelov, and M. Tan. EMMA: End-to-end multimodal model for autonomous driving. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856.
- [6] L. Chen, O. Sinavski, J. Hünermann, A. Karnsund, A. J. Willmott, D. Birch, D. Maund, and J. Shotton. Driving with LLMs: Fusing object-level vector modality for explainable autonomous driving. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 14093–14100. IEEE, 2024.
- [7] J. Mao, Y. Qian, J. Ye, H. Zhao, and Y. Wang. GPT-Driver: Learning to drive with GPT. *arXiv preprint arXiv:2310.01415*, 2023.
- [8] H. Sha, Y. Mu, Y. Jiang, L. Chen, C. Xu, P. Luo, S. Li, M. Tomizuka, W. Zhan, and M. Ding. LanguageMPC: Large language models as decision makers for autonomous driving. *arXiv preprint arXiv:2310.03026*, 2023.
- [9] D. Fu, X. Li, L. Wen, M. Dou, P. Cai, B. Shi, and Y. Qiao. Drive like a human: Rethinking autonomous driving with large language models. In *Winter Conference on Applications of Computer Vision Workshops (WACVW)*, pages 910–919. IEEE, 2024.
- [10] L. Wen, D. Fu, X. Li, X. Cai, T. Ma, P. Cai, M. Dou, B. Shi, L. He, and Y. Qiao. DiLu: A knowledge-driven approach to autonomous driving with large language models. In *International Conference on Learning Representations*, volume 2024, pages 34503–34522, 2024.
- [11] T. Qian, J. Chen, L. Zhuo, Y. Jiao, and Y.-G. Jiang. nuScenes-QA: A multi-modal visual question answering benchmark for autonomous driving scenario. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 4542–4550, 2024.
- [12] X. Zhou, X. Han, F. Yang, Y. Ma, V. Tresp, and A. Knoll. OpenDriveVLA: Towards end-to-end autonomous driving with large vision language action model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 13782–13790, 2026.
- [13] Y. Lin, S. Illing, and M. Althoff. SanDRA: Safe large-language-model-based decision making for automated vehicles using reachability analysis. *arXiv preprint arXiv:2510.06717*, 2025.
- [14] C. Fruhwirth-Reisinger, D. Malić, W. Lin, D. Schinagl, S. Schultze, and H. Possegger. STS-Bench: A spatio-temporal scenario benchmark for multi-modal large language models in autonomous driving. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025.

- [15] Y. Ma, Y. Cao, W. Ding, S. Zhang, Y. Wang, B. Ivanovic, M. Jiang, M. Pavone, and C. Xiao. dVLM-AD: Enhance diffusion vision-language-model for driving via controllable reasoning. *arXiv preprint arXiv:2512.04459*, 2025.
- [16] F. Schmidt, K. Fedurko, M. Enzweiler, and A. Valada. LAD-Drive: Bridging language and trajectory with action-aware diffusion transformers. *arXiv preprint arXiv:2603.02035*, 2026.
- [17] J. Gao, C. Sun, H. Zhao, Y. Shen, D. Anguelov, C. Li, and C. Schmid. VectorNet: Encoding HD maps and agent dynamics from vectorized representation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11525–11533, 2020.
- [18] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang. VAD: Vectorized scene representation for efficient autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8340–8350, 2023.
- [19] W. Wu, X. Feng, Z. Gao, and Y. Kan. SMART: Scalable multi-agent real-time motion generation via next-token prediction. *Advances in Neural Information Processing Systems*, 37: 114048–114071, 2024.
- [20] S. Ettinger, S. Cheng, B. Caine, C. Liu, H. Zhao, S. Pradhan, Y. Chai, B. Sapp, C. R. Qi, Y. Zhou, et al. Large scale interactive motion forecasting for autonomous driving: The Waymo Open Motion Dataset. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9710–9719, 2021.
- [21] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. nuScenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11621–11631, 2020.
- [22] Y. Li, C. Fan, C. Ge, S. Z. Zhao, C. Li, C. Xu, H. Yao, M. Tomizuka, B. Zhou, C. Tang, M. Ding, and W. Zhan. WOMD-reasoning: A large-scale dataset for interaction reasoning in driving. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 34288–34311, 2025.
- [23] H. Shao, Y. Hu, L. Wang, G. Song, S. L. Waslander, Y. Liu, and H. Li. LMDrive: Closed-loop end-to-end driving with large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15120–15130, June 2024.
- [24] S. Wang, Z. Yu, X. Jiang, S. Lan, M. Shi, N. Chang, J. Kautz, Y. Li, and J. M. Alvarez. OmniDrive: A holistic vision-language dataset for autonomous driving with counterfactual reasoning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22442–22452, June 2025.
- [25] M. Liang, B. Yang, R. Hu, Y. Chen, R. Liao, S. Feng, and R. Urtasun. Learning lane graph representations for motion forecasting. In *ECCV*, 2020.
- [26] Z. Zhou, L. Ye, J. Wang, K. Wu, and K. Lu. HiVT: Hierarchical vector transformer for multi-agent motion prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [27] Q. Sun, X. Huang, J. Gu, B. C. Williams, and H. Zhao. M2i: From factored marginal trajectory prediction to interactive prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6543–6552, 2022.
- [28] J. Pillion, X. B. Peng, and S. Fidler. Trajenglish: Traffic modeling as next-token prediction. In *The Twelfth International Conference on Learning Representations*, 2024.
- [29] Z. Zhong, D. Rempe, Y. Chen, B. Ivanovic, Y. Cao, D. Xu, M. Pavone, and B. Ray. Language-guided traffic simulation via scene-level diffusion. In *Conference on Robot Learning*, 2023.

- [30] S. Tan, B. Ivanovic, X. Weng, M. Pavone, and P. Kraehenbuehl. Language conditioned traffic generation. In *7th Annual Conference on Robot Learning*, 2023.
- [31] W.-J. Chang, W. Zhan, M. Tomizuka, M. Chandraker, and F. Pittaluga. LANGTRAJ: Diffusion model and dataset for language-conditioned trajectory simulation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 26622–26631, October 2025.
- [32] S. Tan, B. Ivanovic, Y. Chen, B. Li, X. Weng, Y. Cao, P. Krähenbühl, and M. Pavone. Promptable closed-loop traffic simulation. In *8th Annual Conference on Robot Learning*, 2024.
- [33] A. Felemban, N. Hroub, J. Ding, E. Abdelrahman, X. Shen, A. Mohamed, and M. Elhoseiny. iMotion-LLM: Instruction-conditioned trajectory generation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 2710–2720, 2026.
- [34] B. Liao, S. Chen, X. Wang, T. Cheng, Q. Zhang, W. Liu, and C. Huang. MapTR: Structured modeling and learning for online vectorized hd map construction. In *International Conference on Learning Representations*, 2023.
- [35] J. Gu, C. Sun, and H. Zhao. DenseTNT: End-to-end trajectory prediction from dense goal sets. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 15303–15312, 2021.
- [36] S. Shi, L. Jiang, D. Dai, and B. Schiele. Motion transformer with global intention localization and local movement refinement. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [37] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [38] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [39] P. Gao, J. Han, R. Zhang, Z. Lin, S. Geng, A. Zhou, W. Zhang, P. Lu, C. He, X. Yue, H. Li, and Y. Qiao. LLaMA-Adapter V2: Parameter-efficient visual instruction model. *arXiv preprint arXiv:2304.15010*, 2023.
- [40] H. Liu, C. Li, Y. Li, and Y. J. Lee. Improved baselines with visual instruction tuning. *arXiv preprint arXiv:2310.03744*, 2024.
- [41] Y. Chai, B. Sapp, M. Bansal, and D. Anguelov. MultiPath: Multiple probabilistic anchor trajectory hypotheses for behavior prediction. In *Proceedings of the Conference on Robot Learning*, 2020.
- [42] J. Ngiam, B. Caine, V. Vasudevan, Z. Zhang, H.-T. L. Chiang, J. Ling, R. Roelofs, A. Bewley, C. Liu, A. Venugopal, et al. Scene transformer: A unified architecture for predicting multiple agent trajectories. In *International Conference on Learning Representations*, 2022.
- [43] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li. Planning-oriented autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17853–17862, 2023.

- [44] S. Hu, L. Chen, P. Wu, H. Li, J. Yan, and D. Tao. St-p3: End-to-end vision-based autonomous driving via spatial-temporal feature learning. In *European Conference on Computer Vision (ECCV)*, 2022.
- [45] Z. Huang, H. Liu, and C. Lv. GameFormer: Game-theoretic modeling and learning of transformer-based interactive prediction and planning for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3903–3913, October 2023.

A Tokenization Details

A.1 List of Special Tokens

Table 8 summarizes the additional tokens introduced into VectorLLM vocabulary.

Token	Description
VEC_0 – VEC_1023	Discrete vector tokens for vectorized scene.
AGENT_ID_	Prefix used to identify a specific agent instance.
AGENT_TYPE_Vehicle	Indicates that the agent is a vehicle.
AGENT_TYPE_Pedestrian	Indicates that the agent is a pedestrian.
AGENT_TYPE_Cyclist	Indicates that the agent is a cyclist.
AGENT_TYPE_Other	Indicates an agent type not covered by predefined categories.
AGENT_TYPE_Unset	Indicates that the agent type is unknown or unspecified.
TRAJ_NONE	Marks the absence of trajectory information.
EGO_TRAJ_START	Start delimiter for the ego trajectory sequence.
EGO_TRAJ_END	End delimiter for the ego trajectory sequence.
AGENT_TRAJ_START	Start delimiter for an agent trajectory sequence.
AGENT_TRAJ_END	End delimiter for an agent trajectory sequence.
MAP_START	Start delimiter for map information.
MAP_END	End delimiter for map information.
ROAD_START	Start delimiter for a road segment description.
ROAD_END	End delimiter for a road segment description.

Table 8: Custom tokens added to VectorLLM vocabulary.

A.2 Vectorization Details

Vector preprocessing. We first transform all scene coordinates into the ego-centric coordinate frame of the current scene. The ego vehicle’s current position is translated to the origin, and the ego heading is rotated to align with the positive x -axis. This transformation removes dependence on the global map coordinate system and allows map elements and agent trajectories from different scenes to be represented in a consistent local frame.

After this transformation, both dynamic agent motion and static map geometry are represented as ordered sequences of displacement vectors. For an agent trajectory with valid positions $p_1, \dots, p_T$, where $p_t = (x_t, y_t)$, each motion vector is computed for $t = 2, \dots, T$ as

$$v_t = p_t - p_{t-1} = (\Delta x_t, \Delta y_t).$$

We apply this representation to the ego trajectory history and to surrounding-agent histories. Invalid or missing states are omitted from the serialized sequence, and delimiter tokens are used to separate the ego trajectory from different surrounding-agent trajectories.

For static map geometry, we first filter map elements to keep the local scene context around the ego trajectory. We use an ellipse-based trajectory proximity filter with radius 30.0m and ellipse ratio 1.5. The filter keeps map points that fall inside the trajectory-centered region and removes distant map elements that are unlikely to affect the modeled behavior.

Each retained map polyline is then resampled before tokenization. We use curvature-adaptive map resampling so that high-curvature regions are represented with shorter displacement vectors and near-straight regions are represented with longer displacement vectors. For a polyline point p_i , we estimate local curvature using the turn angle between the adjacent segments $p_{i-1} \rightarrow p_i$ and $p_i \rightarrow p_{i+1}$. Let θ_{turn} denote this turn angle. Each interior point is assigned a target vector length according to θ_{turn} :

$$\ell_i = \begin{cases} 0.5\text{m}, & \theta_{\text{turn}} \geq 60^\circ, \\ 1.0\text{m}, & 30^\circ \leq \theta_{\text{turn}} < 60^\circ, \\ 1.5\text{m}, & 10^\circ \leq \theta_{\text{turn}} < 30^\circ, \\ 2.0\text{m}, & \theta_{\text{turn}} < 10^\circ. \end{cases}$$

The first and last points inherit the target length assigned to their nearest interior neighbor. Consecutive points with the same target length are grouped into sub-polylines, and each sub-polyline is

resampled using its assigned target length. If the sub-polyline length is not an integer multiple of the target length, a final shorter remainder vector is added. After resampling, each map vector is the displacement between two adjacent resampled points, using the same vector format as agent motion.

Vector token vocabulary construction. We construct a discrete vector vocabulary before converting displacement-vector sequences into token sequences. The vocabulary is built from one-step displacement vectors collected from the WOMD training split. Each candidate vector is represented as

$$a_i = (\Delta x_i, \Delta y_i) \in \mathbb{R}^2.$$

We construct the vector vocabulary using the k-disks procedure from Trajenglish [28]. At a high level, k-disks selects representative displacement vectors by repeatedly choosing a candidate vector, covering nearby candidates within a distance tolerance, removing the covered set, and continuing until the target vocabulary size is reached.

Trajenglish compares k-disks with alternative vocabulary-construction methods, including grid-based tokenization and k-means. Their conclusion is that k-disks gives the strongest discretization quality among the tested methods. Grid-based tokenization allocates templates uniformly over the discretization range, which wastes vocabulary entries in rarely observed motion regions. K-means adapts better to the data distribution, but Trajenglish reports that it still has higher expected discretization error and more large-error cases than k-disks. These findings motivate our use of k-disks for selecting representative displacement vectors.

We use the same k-disks procedure with adjusted parameters for our vectorization setting. Specifically, we evaluate vocabulary sizes $K \in \{384, 512, 1024, 2048\}$, where 384 and 512 follow the original Trajenglish setting, and 1024 and 2048 test whether a larger vocabulary improves reconstruction accuracy. We select $K = 1024$ because it achieves mean pointwise reconstruction error below 0.03m on WOMD road polylines, while increasing the vocabulary size to 2048 does not further improve reconstruction error. Each selected representative vector is assigned one vector-token ID, producing a shared scene vocabulary for both dynamic agent motion and static map geometry.

Vector token assignment. The continuous displacement vectors are discretized by assigning them to entries in the vector vocabulary. Each vocabulary vector corresponds to one vector-token ID, and tokenization maps each continuous vector sequence to a sequence of vector-token IDs.

For long vector sequences, especially static map polylines, independent nearest-vector assignment can accumulate reconstruction drift. To reduce this error, we use beam-search token assignment. Given a raw trajectory or map polyline, we first convert it into a sequence of displacement vectors. To reduce the search space, we preselect the top 40 candidate vocabulary vectors according to their minimum ℓ_2 distance to any ground-truth displacement vector in the sequence.

Beam search is then performed over this candidate set. The initial state is the first point of the raw trajectory or polyline. At each timestep, every beam is expanded by adding each candidate displacement vector to the current reconstructed position. The step cost is the Euclidean distance between the reconstructed next point and the corresponding ground-truth next point. The cumulative cost is normalized by the number of elapsed timesteps. After each timestep, only the 10 beams with the lowest normalized cumulative cost are retained. The final token sequence is the vector-token sequence from the lowest-cost complete beam.

Finally, tokenized map and motion elements are serialized using semantic marker tokens and delimiter tokens. These tokens indicate whether a vector-token sequence corresponds to map geometry, ego history, surrounding-agent history, or task-specific output positions. The serialized scene tokens are concatenated with the language prompt to form the LLM input context. During decoding, selected vector tokens are mapped back to their vocabulary vectors and reconstructed into metric coordinates by cumulative summation.

A.3 Examples of LLM Input Context

To illustrate the input format used by our model, we provide a shortened representative example below. The full input consists of a serialized driving scene containing map elements, the ego-vehicle history, and surrounding-agent histories. For readability, we omit most repeated vector tokens and replace long subsequences with `...`. The example is intended to show the structure of the representation rather than a complete scene.

```
MAP_START
  ROAD_START
    START_[x_1, y_1]
    LaneCenter-SurfaceStreet
    VEC_a VEC_b VEC_c ...
  ROAD_END

  ROAD_START
    START_[x_2, y_2]
    RoadEdgeBoundary
    VEC_d VEC_e VEC_f ...
  ROAD_END

  ROAD_START
    START_[x_3, y_3]
    Crosswalk
    VEC_g VEC_h VEC_i ...
  ROAD_END
MAP_END

EGO_TRAJ_START
  START_[0.0, 0.0]
  AGENT_TYPE_Vehicle
  VEC_0 VEC_0 VEC_0 VEC_k ...
EGO_TRAJ_END

AGENT_TRAJ_START
  START_[x_4, y_4]
  AGENT_ID_<agent_1>
  AGENT_TYPE_Vehicle
  VEC_m VEC_n VEC_o ...
AGENT_TRAJ_END
```

The tokens `START_[x,y]` encode the local starting position of each element, while `VEC_*` tokens denote discretized motion or geometry vectors. Map entities are grouped between `MAP_START` and `MAP_END`; each road or crosswalk element is enclosed by corresponding start and end markers. Similarly, ego and non-ego agent histories are represented as trajectory blocks with an agent identifier, type, and sequence of vector tokens.

B Training Data Generation Details

B.1 WOMD QA

Given a WOMD scene, we generate QA supervision using a deterministic extraction pipeline over the ego vehicle trajectory, surrounding-agent trajectories, and local map geometry. The pipeline operates on the ego-centered scene representation and extracts structured scene facts, including target-vehicle motion, nearby-agent attributes, map context, relative spatial relations, heading directions, and goal-oriented motion cues.

The extracted facts support several groups of question types:

1. **Motion-behavior questions.** These questions ask about the ego vehicle’s motion pattern, including speed-up and slow-down events, stopping, constant-speed motion, left and right turns, left and right lane changes, lane keeping, following, and yielding. *Example: Does the ego vehicle slow down in the scene?*

2. **Map-context questions.** These questions ask about the relationship between the ego vehicle and nearby map elements, including road curving direction, intersections, stop-sign encounters, lanes, and road boundaries. *Example: Is the ego vehicle approaching an intersection?*
3. **Spatial-reasoning questions.** These questions ask about relative positions, distances, and heading directions of surrounding agents with respect to the ego vehicle. *Example: Is there a vehicle ahead of the ego vehicle in the same lane?*
4. **Counting questions.** These questions ask for the number of agents satisfying specific conditions, such as agent type, distance range, relative location, heading direction, or driving behavior. *Example: How many vehicles are within 20 meters of the ego vehicle?*
5. **Goal-oriented questions.** These questions ask about the ego vehicle’s general intent or likely maneuver based on its trajectory and local map context. *Example: What is the ego vehicle likely to do?*

For temporally localized behaviors, each detected behavior is represented as a semantic event with a start time and end time. Events of the same type that overlap or occur within a short temporal gap are merged, and very short events are discarded to reduce noisy labels. The resulting event sequence provides a compact symbolic summary of the ego vehicle’s motion, map context, and interactions.

Before constructing QA pairs, we first convert each driving scene into a set of symbolic facts and events. Symbolic facts refer to structured scene attributes extracted from the vectorized map and agent trajectories, such as agent type, relative position, distance to the ego vehicle, heading direction, nearby lane or intersection context, and whether a relevant map element is present. Symbolic events refer to time-indexed driving behaviors or scene conditions, such as slowing down, stopping, turning, lane changing, following, yielding, entering an intersection, or encountering a stop sign. Each event is represented with its event type and the time interval over which it occurs. These symbolic facts and events provide an explicit intermediate representation from which question-answer supervision can be generated.

The symbolic facts and events are converted into QA pairs using a fixed set of query templates. For direct behavior questions, the answer states whether the corresponding event occurs and, if so, lists the relevant time intervals in chronological order. For interaction questions, the procedure checks temporal co-occurrence between event categories within predefined windows. For example, speed adjustment on curves is detected when acceleration or deceleration overlaps with, or occurs near, a road-curve interval; stopping or turning at intersections is detected by matching stop or turn intervals to intersection or stop-sign intervals; and speed adjustment while following or yielding is detected by matching speed-change events to following or yielding intervals. For spatial and counting questions, the answer is computed directly from the extracted relative positions, headings, distances, agent types, and behavior labels. If the prerequisite scene condition is absent, the answer explicitly states that the condition does not occur; otherwise, it gives a concise response with the relevant symbolic or temporal evidence.

To avoid training only on rigid template language, we use Qwen3-4B-Instruct-2507 to convert the extracted QA templates into natural-language supervision. The input to Qwen consists of the deterministic question template, the extracted symbolic facts, event labels, and temporal evidence. Qwen is instructed to paraphrase the question and answer into fluent language while preserving the original event type, yes/no label, counts, relative relations, and time intervals. It is not used to infer new driving behaviors or scene geometry from raw data; all labels and evidence are produced by the rule-based extraction pipeline. The resulting natural-language QA pairs are used as supervision for the WOMD QA training task.

B.2 Chain-of-thought Synthesis

We generate chain-of-thought targets during data preparation using a rule-based scene analysis pipeline followed by LLM-based verbalization. The pipeline follows the same general structure as the WOMD QA construction: it first extracts symbolic facts from the vectorized scene, and then

converts these facts into natural-language supervision. The purpose of this process is to make the reasoning trace explicitly grounded in the map and motion elements that appear in the serialized input context.

We then apply a set of high-level driving-scene rules to identify salient constraints and interactions. These rules are not intended to enforce traffic laws; instead, they encode common behavioral patterns used to interpret how agents move through a scene, such as lane following, yielding, proximity, relative motion, potential conflicts, stop-sign or intersection relevance, and maneuver choices.

For trajectory-generation examples, we additionally detect future valid lanes and feasible driving options. The rule-based module searches the local road topology around the ego vehicle and identifies lane segments that are reachable from the current ego position and consistent with the ego’s current heading and motion state. These candidate lanes are used to infer feasible maneuvers such as continuing along the current lane, turning, changing lanes, slowing down, stopping, or yielding. We also compare these driving options with the future ego trajectory to identify the intended maneuver and the supporting map context. This provides the symbolic supervision used to explain why a future goal or trajectory is feasible.

The extracted symbolic facts are then provided to an off-the-shelf LLM to synthesize natural-language reasoning traces. Each trace is generated from the structured facts available for that specific scene and task. We do not ask the LLM to infer scene geometry, agent interactions, or feasible maneuvers from raw trajectories. Instead, the rule-based extraction first determines the relevant map and motion evidence, and the LLM only rewrites this evidence into a natural-language reasoning trace.

The trace-generation workflow is:

1. Extract structured map and motion facts from the vectorized scene, including ego motion, nearby agents, relative positions, map context, and detected behavior events.
2. Apply high-level driving-scene rules to select the evidence relevant to the current task, such as lane following, yielding, following another agent, approaching an intersection, or making a turn.
3. Provide the selected facts to an off-the-shelf LLM with a fixed prompt that asks it to verbalize the evidence in a driving-oriented reasoning format.
4. Use the generated trace as supervision before the final answer, goal, or trajectory output.

The generated trace follows a common structure: it first identifies the ego motion state, then selects relevant map context and nearby agents, then reasons about interactions or feasible maneuvers, and finally supports the task-specific output. The exact content of the trace depends on the task. For example:

1. **QA trace.** For a question such as “Does the ego vehicle slow down before reaching the intersection?”, the extracted facts may indicate that the ego vehicle decelerates during a specific time interval and that the same interval overlaps with an approaching-intersection interval. The generated trace states that the vehicle is approaching the intersection, observes that its speed decreases before or during that approach, and supports the final answer that the vehicle slows down before reaching the intersection.
2. **Trajectory-generation trace.** For a trajectory-generation example, the extracted facts may indicate that the ego vehicle is following the current lane, that a nearby vehicle is ahead, and that the road remains straight. The generated trace explains that the ego should continue along the lane while maintaining the interaction context with the leading vehicle, and supports a trajectory that follows the lane direction.

This procedure grounds the reasoning trace in the vectorized map and motion elements used by VectorLLM while expressing the supervision in language form compatible with the pretrained LLM backbone. At inference time, VectorLLM generates the reasoning trace and final output itself. During training, the chain-of-thought segment is optimized with next-token cross-entropy and precedes

either the final QA answer or the goal-prediction and trajectory-generation target, depending on the task.

C Evaluation Details

C.1 Counterfactual Scene Edit

We construct counterfactual scene-edit examples using a pipeline similar to the WOMD QA and CoT data-generation procedures. Instead of producing question-answer labels or reasoning traces, the pipeline identifies scenes where an edit in the scene is expected to change the ego vehicle’s future behavior. Each example consists of an original WOMD scene, an edited scene, an edit type, and a counterfactual target behavior used for evaluation.

The scene-edit pipeline first extracts structured facts from the ego trajectory, surrounding-agent trajectories, and local map geometry. These facts include ego speed profile, acceleration pattern, lane position, future lane choice, nearby-agent positions and motions, following or yielding relations, and map topology around the ego vehicle. Based on these facts, rule-based filters select scenes where a targeted edit has a clear behavioral interpretation. We consider four edit categories: ego-history edits, agent removal, agent insertion, and lane-geometry edits.

Ego-history edits. Ego-history edits modify the observed past motion of the ego vehicle while keeping the surrounding scene fixed. These edits are designed to test whether the model conditions its future prediction on the ego vehicle’s recent motion state. We construct edits that change the velocity or acceleration trend of the ego history, including slow-to-fast, fast-to-slow, acceleration-to-deceleration, and deceleration-to-acceleration transformations. For example, a slowly moving ego history may be edited into a faster approach profile, or an accelerating history may be edited into a decelerating one. The edited history changes the implied driving intent or dynamic state while preserving the local map context and surrounding-agent configuration.

Agent-removal edits. Agent-removal edits remove a surrounding vehicle whose presence is likely to constrain the ego vehicle’s future motion. The rule-based pipeline identifies important agents using interaction cues such as proximity, relative position, relative speed, and temporal overlap with the ego vehicle’s behavior. A typical case is a lead vehicle in front of the ego vehicle: if the ego vehicle is following the lead vehicle and slowing down, removing that vehicle creates a counterfactual scene with more free space ahead. The expected counterfactual behavior is that the ego vehicle can maintain speed or move faster than in the original scene. These edits test whether the model uses surrounding-agent interactions rather than only extrapolating the original ego trajectory.

Agent-insertion edits. Agent-insertion edits add a surrounding vehicle that creates a new interaction constraint. The inserted agent is placed in a behaviorally relevant region of the scene, with motion chosen to make it potentially conflict with the ego vehicle’s intended maneuver. For example, if the ego vehicle is turning, an oncoming vehicle may be inserted along the conflicting path so that the ego vehicle should slow down or yield to avoid a collision. These edits test whether the model responds to newly introduced interaction constraints and adjusts the ego trajectory accordingly.

Lane-geometry edits. Lane-geometry edits modify the local map topology to change the set of feasible ego maneuvers. In the main setting, we construct edits that prevent the original turning behavior by adding a road boundary and removing or masking the original turning lane. This creates a counterfactual scene in which the ego vehicle is forced to continue straight instead of following the original turn. These edits test whether the model grounds trajectory generation in the edited map geometry rather than memorizing or copying the original future trajectory.

Evaluation waypoint selection. For each edited scene, we select three future waypoints that the ego vehicle is expected to reach if it responds to the edit in the intended direction. These waypoints are determined from the edit type, the behavior induced by the edit, and the ego vehicle’s current mo-

tion state. The three waypoints are ordered temporally and are used only for evaluation, measuring whether the generated trajectory changes toward the intended edited behavior.

Evaluation. During evaluation, each model receives the edited scene as input and generates multiple future trajectory hypotheses. Each edited scene is associated with three temporally ordered counterfactual waypoint goals. A waypoint is counted as hit if at least one generated trajectory passes within a 1.0m radius of that waypoint. The goal-hit score for a scene is the fraction of its waypoint goals that are hit. We report the average goal-hit score across all edited scenes, which measures how often the generated trajectories reach the intended counterfactual targets.

C.2 VectorLLM+MapTR Variant Implementation Details

VectorLLM normally uses annotated HD-map elements as the vectorized scene input. To evaluate the model under perception-generated map inputs, we construct a VectorLLM+MapTR variant by placing MapTR in front of VectorLLM. MapTR takes multi-view camera images as input and predicts vectorized map elements. These predicted map vectors are then used as the map input to VectorLLM, replacing the annotated HD-map polylines.

The rest of the VectorLLM pipeline is kept unchanged. The MapTR-predicted map elements are processed by the same map vectorization, filtering, tokenization, and serialization procedures used for HD-map inputs.

C.3 MTR & M2I

MTR. We use the official MTR implementation and retrain an MTR model on the WOMB training split using its standard data preprocessing and training configuration. We do not modify the model architecture. For our scene-edit evaluation, each edited WOMB scenario is converted to the same input format expected by MTR, with the edited ego vehicle used as the prediction target. At inference time, MTR produces six future trajectory hypotheses for the ego vehicle.

M2I. We use the released M2I vehicle-vehicle relation, marginal prediction, and conditional prediction checkpoints without retraining. To adapt our scene-edited Waymo samples to the public M2I checkpoint, we convert each edited scenario into M2I’s `tf.train.Example` format and rewrite it as a two-agent pair-interest example. The edited ego vehicle is placed in the first target slot. The second target is selected as the other object of interest if specified in the “track to predict” field; otherwise, we choose the closest valid vehicle to the ego. Only these two vehicles are marked in `state/tracks_to_predict` and `state/objects_of_interest`. We then run the standard M2I relation, marginal, and conditional prediction pipeline, keep only the ego predictions, and convert them to six 80-step future trajectories in Waymo world coordinates for our scene-edit evaluation.

C.4 Counterfactual Language Instruction

We construct a language conditioned trajectory generation subset derived from nuScenes containing scenes with alternative feasible maneuvers. A rule-based scene analysis script identifies behaviorally meaningful alternatives, such as continuing along the current lane, choosing a different lane, yielding, turning, or change of speed. These structured alternatives are then written into natural-language driving commands by an off-the-shelf LLM. Each example contains the original scene, the counterfactual language instruction, and the instruction-selected alternative goals.

CTG++. We use CTG++ as the language-conditioned diffusion baseline without modifying its model architecture or training procedure. CTG++ converts language instructions into differentiable guidance losses for test-time diffusion sampling. We use GPT-5.4-mini to generate the guidance loss. Although CTG++ supports multi-agent instructions, we only control the ego vehicle in our evaluation and map it to `vehicle 1`. Example instructions include “vehicle 1 should turn left at the intersection,” “vehicle 1 should keep moving along the current lane,” and “vehicle 1 should stop.” The generated ego trajectory is evaluated with the same goal-hit metric as VectorLLM.

iMotion-LLM. iMotion-LLM was originally designed for Waymo-format inputs, so we convert nuScenes trainval data into the Waymo-style format expected by its public codebase and attach scene-level language instruction templates. We resample nuScenes to 10 Hz and use 11 history steps plus 80 future steps per sample. For samples near scene boundaries, missing trajectory points are padded by linear extrapolation and marked in the masks. We then train the GameFormer scene encoder on the processed nuScenes data and use its checkpoint to train iMotion-LLM with the original training code. We do not modify the iMotion-LLM architecture; the main adaptation is the nuScenes-to-Waymo-style data conversion.

C.5 CoT Ablation Details

We evaluate the effect of chain-of-thought quality by constructing a corrupted-CoT ablation. The goal of this ablation is to test whether performance gains come from accurate intermediate reasoning rather than simply from adding extra language tokens before the final answer or trajectory target.

To construct corrupted CoT targets, we modify the structured facts before they are passed to the off-the-shelf LLM for reasoning synthesis. For each training example, we randomly corrupt 50% of the extracted facts. The corruption is applied according to the type of each fact. Numeric values, such as distances, speeds, headings, and time intervals, are replaced with randomly generated values. Vector-token references are replaced with randomly sampled vector tokens. Semantic labels, such as road types and agent types, are replaced with randomly sampled labels from the corresponding semantic category. This keeps the corrupted facts syntactically valid while making them inconsistent with the actual scene.

The corrupted fact set is then provided to the same CoT synthesis procedure used for the full model. The off-the-shelf LLM verbalizes these corrupted symbolic facts into natural-language reasoning traces, while the final task target remains unchanged. As a result, the model receives a reasoning trace that has the same format and approximate length as the original CoT supervision but contains incorrect or irrelevant scene evidence.

This ablation isolates the effect of grounded reasoning content. If CoT only helps by increasing the number of intermediate tokens, corrupted CoT should perform similarly to accurate CoT. In contrast, the performance drop observed with corrupted CoT indicates that the correctness of the extracted scene facts and their alignment with the serialized vector tokens are important for both QA and trajectory generation.

C.6 Additional Qualitative Analysis

C.6.1 Latent-space Alignment via t-SNE

We provide an additional t-SNE visualization in Fig. 5 to further examine how language tokens and vector tokens align in the model’s latent space. This analysis uses contextual hidden states produced by the model for specific language-conditioned trajectory-generation examples. We aggregate examples from multiple scenes involving right turns near intersections and/or stop signs, where intersection and stop-sign context may overlap.

For each example, we select language tokens corresponding to relevant semantic concepts, such as “turn”, “right”, “intersection”, and “stop sign”. We also select scene-relevant vector tokens whose map or trajectory meaning corresponds to these language tokens. The hidden state for each selected token is computed by averaging its contextual representation from

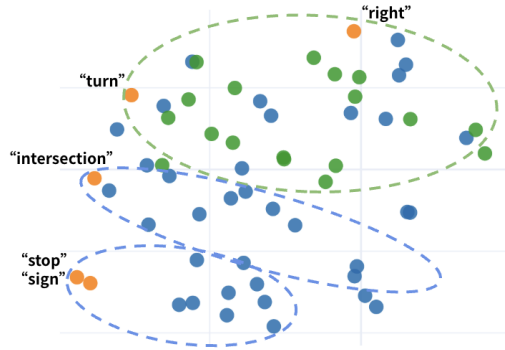


Figure 5: Additional t-SNE visualization of contextual hidden states from language-conditioned trajectory-generation examples involving right turns near intersections and/or stop signs. Orange points denote selected language tokens, blue points denote map vector tokens, and green points denote trajectory vector tokens.

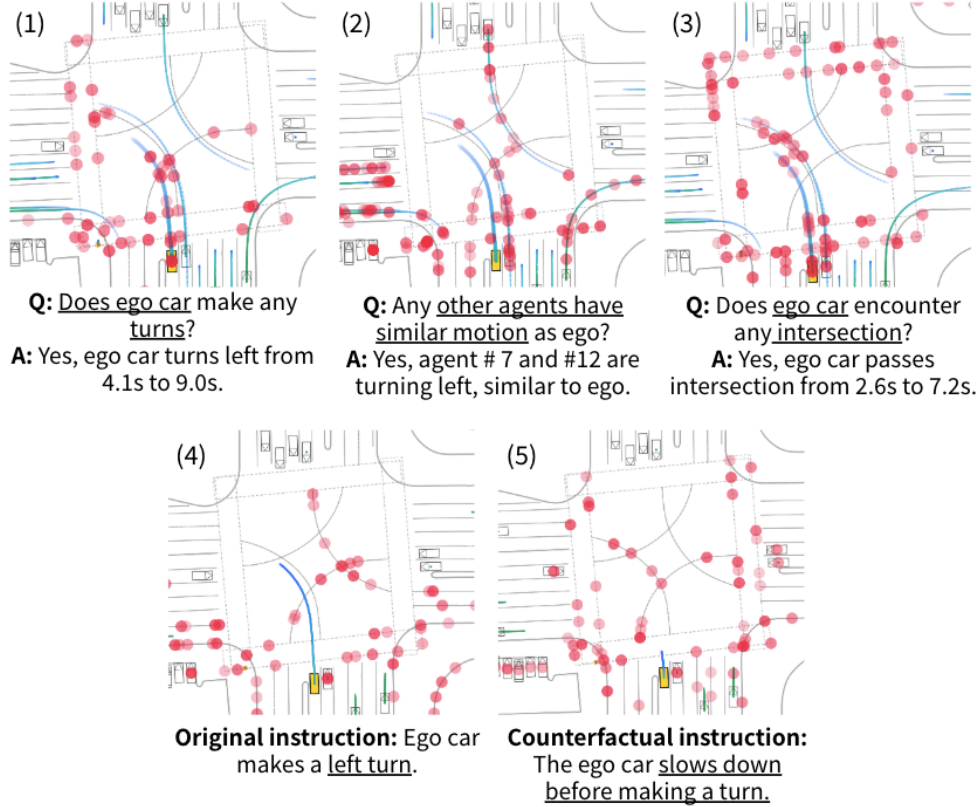


Figure 6: Additional attention-based grounding examples. Red dots indicate selected vector tokens attended by the underlined language tokens in the question or instruction. The ego vehicle is shown in yellow.

model layers 18 to 24, excluding the final output layer. We then apply t-SNE to project these hidden states into two dimensions.

The visualization shows that language tokens are located near vector tokens with related driving meaning. The language tokens “turn” and “right” appear close to trajectory vector tokens associated with the right-turn maneuver, suggesting that the model places maneuver-related language near motion tokens that express the corresponding behavior. In contrast, “intersection” and “stop sign” are positioned closer to map vector tokens, reflecting their stronger association with static scene context rather than the trajectory itself.

This pattern is consistent with the intended role of the vector-token interface. Since map geometry, agent motion, and language prompts are processed in a unified token sequence, the model can learn contextual relationships between linguistic concepts and the vector tokens that express the corresponding driving-scene evidence. The visualization also suggests a separation between maneuver semantics and map-context semantics: turning-related language is closer to trajectory tokens, while intersection- and stop-sign-related language is closer to map tokens.

C.6.2 Attention-based Grounding Visualization

Attention extraction and visualization. We provide additional qualitative attention visualizations in Fig. 6 to examine how VectorLLM connects language tokens with vectorized scene elements. For each example, we select the language tokens corresponding to the underlined word or phrase in the question or instruction. We extract the attention weights from these language tokens to the serialized vector tokens and average the attention across the last 10 model layers. For each selected language

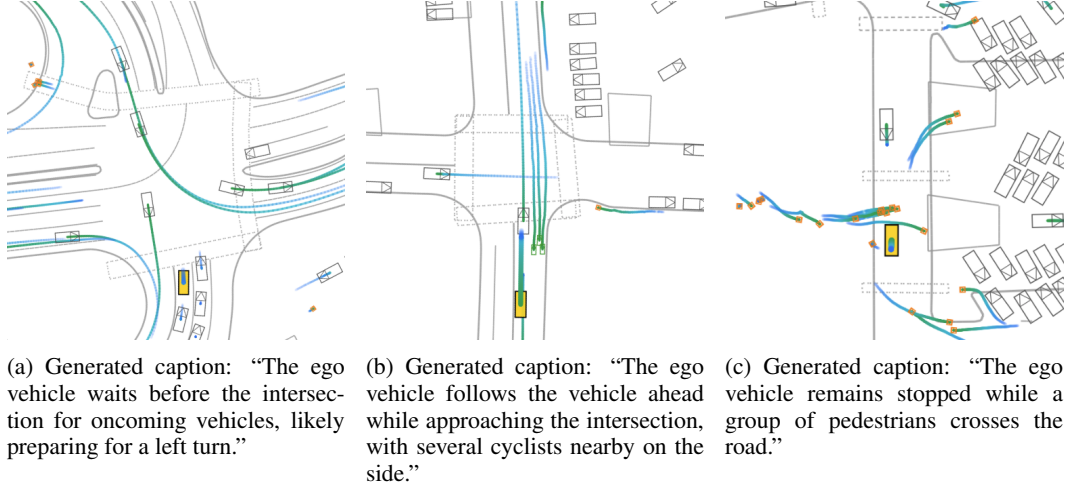


Figure 7: Additional qualitative examples for the unseen scene-captioning task.

token, we aggregate its attention over vector tokens and keep the top-attended vector tokens. The union of these selected vector tokens is then visualized in the scene.

Each visualized red dot corresponds to one selected vector token. To place the dot in metric space, we track the position of each vector token when reconstructing the serialized map polylines and agent trajectories back into the ego-centric coordinate frame. The opacity of each red dot is determined by normalizing its attention value using the minimum and maximum attention values among all visualized tokens in the same example. These attention maps are intended as qualitative evidence of grounding behavior rather than a definitive causal attribution of the model’s prediction.

Discussion of examples. Fig. 6(1)–(3) show additional QA examples. In Fig. 6(1), the question asks whether the ego car makes any turns. The attention associated with the underlined word “turns” concentrates on the curved road region and the ego path where the left-turn behavior occurs, supporting the answer that the ego car turns left. In Fig. 6(2), the question asks whether other agents have motion similar to the ego. The attended vector tokens highlight nearby agent trajectories with similar turning behavior, which is consistent with the answer identifying other agents that are also turning left. In Fig. 6(3), the question asks whether the ego car encounters an intersection. The attention shifts toward the intersection region and nearby road structure, supporting the answer that the ego car passes through an intersection during the specified time interval.

Fig. 6(4)–(5) compare language-conditioned trajectory generation under an original instruction and a counterfactual instruction. In Fig. 6(4), the original instruction asks the ego car to make a left turn while driving through the intersection. The attention focuses on the turning path and the road geometry associated with the intended maneuver. In Fig. 6(5), the counterfactual instruction asks the ego car to slow down before making a turn. The visualization shows attention not only to the turning context, but also to the approach region and nearby scene elements relevant to slowing down before the maneuver. This matches the generated behavior: the trajectory continues to preserve the original turning intention, but moves slowly and only begins the turn near the end of the predicted trajectory horizon. Together, these examples suggest that VectorLLM can attend to different vectorized scene elements depending on the language query or instruction, supporting both scene-grounded QA and counterfactual language-conditioned generation.

C.7 Additional Unseen Task: Scene Captioning

Discussion. We include additional qualitative examples of the unseen scene-captioning task in Fig. 7. This task is not part of the supervised training tasks. Instead, it is evaluated by prompting VectorLLM with the same chain-of-thought QA-style template used elsewhere, while asking the model to describe the driving scene and include relevant motion, interaction, and map information.

Table 9: Ablation of LLM backbone family and model size on counterfactual language-instruction controllability. All models use the same VectorLLM vector-token scene representation and are evaluated on the nuScenes counterfactual language-instruction benchmark using the full training split.

Backbone	Size	Adaptation	Avg.↑	@1s↑	@2s↑	@3s↑
Llama-3.2-Instruct	1B	Full	67.2	85.0	65.5	51.0
Gemma-3	270M	Full	44.2	69.5	41.0	22.0
Gemma-3	1B	Full	73.3	90.5	70.5	59.0
Gemma-3	4B	Full	76.5	92.0	74.0	63.5
Gemma-3	12B	Full	81.0	94.0	78.5	70.5
Gemma-3	27B	LoRA	72.8	89.5	68.0	61.0
Qwen3	0.6B	Full	75.4	91.0	73.5	62.0
Qwen3	1.7B	Full	77.2	92.0	75.0	64.5
Qwen3	4B	Full	77.7	92.5	76.5	64.0
Qwen3	8B	Full	80.5	93.5	80.5	67.5
Qwen3	14B	Full	82.2	95.0	81.5	70.0
Qwen3	32B	LoRA	70.7	86.5	65.0	60.5

Table 10: Ablation of training data size on counterfactual language-instruction controllability. The experiment uses Qwen3-8B with full fine-tuning and the same VectorLLM vector-token scene representation.

Data split	Avg.↑	@1s↑	@2s↑	@3s↑
20%	69.2	80.5	66.5	60.5
50%	73.8	86.0	73.5	62.0
100%	80.5	93.5	80.5	67.5

The model first generates a reasoning trace and then produces a final scene description. We treat these examples as qualitative evidence of generalization rather than as a formally benchmarked captioning result.

The examples suggest that the learned vector-language interface can support scene-level description beyond the tasks used for supervision. In Fig. 7(a), the generated caption identifies that the ego vehicle is waiting before an intersection and relates this behavior to the likely intention of preparing for a left turn. This indicates that the model connects ego motion, road geometry, and the presence of oncoming traffic into a coherent high-level description. In Fig. 7(b), the generated caption describes the ego vehicle as following a vehicle ahead while proceeding to the intersection, and also mentions nearby cyclists. This example shows that the model can summarize both the ego vehicle’s motion and the surrounding multi-agent context. In Fig. 7(c), the generated caption states that the ego vehicle remains stopped while pedestrians cross the road, correctly capturing an interaction between the ego vehicle and other road users.

Overall, these examples suggest that VectorLLM can reuse the same grounded vector-token representation not only for question answering and trajectory generation, but also for open-ended scene description. Although the captions are not always perfectly phrased and we do not evaluate them with a dedicated captioning metric, they show that the model can organize map context, agent motion, and interaction cues into concise natural-language summaries without explicit caption-training supervision.

C.8 Ablation: Other LLM Backbones

This ablation leads to three main observations. First, increasing the LLM backbone size can improve counterfactual instruction controllability, but the effect is limited after moderate model scale.

Among fully fine-tuned Gemma-3 models, performance improves from 76.5 Avg. at 4B to 81.0 Avg. at 12B. Similarly, Qwen3 improves from 80.5 Avg. at 8B to 82.2 Avg. at 14B. These results indicate that larger pretrained backbones can help VectorLLM interpret counterfactual instructions, but the observed gains are relatively modest as model size increases.

Second, training data size has a more pronounced effect. As shown in Table 10, Qwen3-8B improves from 69.2 Avg. with 20% data to 73.8 Avg. with 50% data and 80.5 Avg. with the full training split. The improvement is consistent across all horizons, suggesting that additional counterfactual instruction supervision improves both short-horizon compliance and longer-horizon controllability.

Third, the largest-backbone results should be interpreted separately from the fully fine-tuned scaling trend. Gemma-3-27B and Qwen3-32B are trained with LoRA due to hardware constraints. Their lower performance does not imply that larger backbones are inherently worse for VectorLLM. Rather, these results suggest that LoRA may provide insufficient adaptation capacity for this vector-token controllability setting. With full fine-tuning, these larger models may achieve stronger performance, so we avoid drawing a direct conclusion about large-model scaling from the LoRA rows.

Overall, the ablation suggests that the most reliable direction for improving VectorLLM controllability is to increase the amount and diversity of counterfactual instruction data. Backbone scaling provides some benefit, but the data-size ablation shows a clearer and more consistent improvement.

C.9 Architectural comparison with iMotion-LLM

iMotion-LLM [33] is closely related to VectorLLM because both methods combine vectorized driving-scene information, natural-language instructions, and trajectory generation. However, the two methods use the LLM in different parts of the prediction pipeline.

In iMotion-LLM, the driving scene is first processed by a trajectory-prediction backbone based on GameFormer [45]. The scene encoder converts the vectorized driving scene into continuous scene features. These features are then projected into the LLM input space, where the LLM incorporates language information through special scene and output embeddings. The resulting LLM output embeddings are mapped back to the trajectory-model embedding space and passed to the trajectory decoder to generate future trajectories. Under this design, the LLM functions as a language-conditioning module inside an external trajectory-prediction pipeline, while the encoder and decoder of the trajectory backbone remain responsible for representing the scene and producing the trajectory.

VectorLLM follows a different design. Instead of using a separate trajectory encoder to convert the scene into continuous embeddings, VectorLLM discretizes map geometry, ego history, surrounding-agent histories, and trajectory outputs into vector tokens. These vector tokens are inserted directly into the LLM context together with the language prompt. The LLM therefore receives the scene as an explicit token sequence, rather than through projected scene embeddings produced by a separate encoder. Future trajectories are also generated as vector-token outputs and decoded back into metric coordinates by cumulative summation.

This architectural difference changes the role of the LLM. In iMotion-LLM, the LLM mainly injects language information into a trajectory model that already has its own scene encoder and trajectory decoder. In VectorLLM, the LLM is the central model that processes the vectorized scene, language prompt, reasoning trace, and trajectory tokens within a unified token sequence. This design removes the separate scene encoder, scene-to-LLM projection module, and trajectory decoder used in encoder-decoder trajectory pipelines, and makes map and motion elements directly token-addressable by the LLM.